

DS11 — Content-Based + Collaborative Hybrid Recommender System

Final Project Plan • Workflow • File Structure • Dataset Decisions

Environment: Anaconda (local, 6GB VRAM) | Colab (e5 encoding only, one-time)

1. Final Dataset Decisions

All decisions below are final after full analysis of the Amazon Reviews 2023 dataset statistics, VRAM constraints, and DS11 requirements.

1.1 Single dataset — Amazon Reviews 2023 (3 categories only)

Do NOT use MovieLens. Do NOT use random sampling.

MovieLens has no review text or product metadata — it kills TF-IDF, T5, e5-base-v2, semantic search, content-based scoring, and Apriori. All 12 project files require Amazon. Random sampling (previous approach) destroys user history — CF, ALS, SVD++ all need users with multiple interactions and cannot learn from a randomly sliced dataset.

Category	Total reviews	Chunk loaded	After user-filter	Why this category
Video_Games	4.6M	100K	~60-70K	Tech buyers, rich review text, cross-sells with instruments and software
Musical_Instruments	3.0M	100K	~55-65K	Passionate buyers write long technical reviews — excellent for NLP. Same tech demographic.
Software	4.9M	100K	~65-75K	Longest, most opinionated reviews in the dataset. Best TF-IDF vocabulary and T5 signal.
Combined total	~12.5M	300K raw	~180-200K	After merge + user-activity filter (≥5 interactions). Fits in 4-6GB RAM.

1.2 Why these 3 categories beat 7 unrelated ones

Category count affects each model type differently. For CF models (ALS, MF, NCF, item-item), fewer focused categories is better — not worse. Here is why:

Model type	3 focused categories (Video_Games + MI + Software)	7 unrelated categories (previous approach)
ALS / MF / NCF / item-item CF	Dense matrix — same tech buyers overlap across all 3. CF learns strong item neighbors.	Sparse matrix — a baby products buyer never overlaps a software buyer. CF learns nothing.
Apriori market-basket	Real cross-category rules appear: game + controller, guitar + cable + software.	Isolated blocks — fashion basket never intersects appliances basket.
TF-IDF + T5 + e5-base-v2	Software + MI reviews are the longest in the dataset. Best NLP signal.	Health + Baby reviews are shortest and least opinionated. Weakest NLP signal.
Content-based + semantic search	3-domain scope is acceptable. State it as a known limitation in the A/B notebook.	More domains but CF is broken — hybrid system cannot function.

1.3 The parent_asin fix — most critical change in the entire codebase

Your previous merge was wrong — it created fake duplicate items and destroyed CF.

Old code: `review_df.merge(meta_df, left_on='asin', right_on='parent_asin')`. The review asin is a variant-level ID (red 32GB version). One product with 20 color variants appeared as 20 different items in your CF matrix — inflating the item space from ~40K real products to 200K+ fake variants. This made the user-item matrix 5x sparser than it should be, and all CF models failed silently.

The fix — use `parent_asin` from the review side for the merge and for all downstream CF work:

```
# WRONG — previous code
merge_df = review_df.merge(meta_df, left_on='asin', right_on='parent_asin',
suffixes=('_rev', '_meta'))

# CORRECT — use parent_asin from reviews as the join key
merge_df = review_df.merge(meta_df, left_on='parent_asin', right_on='parent_asin',
suffixes=('_rev', '_meta'))

# And for ALL CF work downstream — use parent_asin, never asin
# user_id + parent_asin + rating + timestamp are your CF triplets
# parent_asin = product-level ID (38-47K unique items)
# asin         = variant-level ID (200K+ — do not use for CF)
```

1.4 How to filter — user-activity filter, not random sample

Replace the previous `merge_df.sample(50000)` with this. It keeps far more data and ensures every user has real interaction history:

```
CATEGORIES = ['Video_Games', 'Musical_Instruments', 'Software']
CHUNK_SIZE = 100_000 # first 100K rows per category

# After loading and merging all 3 categories:
user_counts = merge_df['user_id'].value_counts()
```

```
active_users = user_counts[user_counts >= 5].index
merge_df      = merge_df[merge_df['user_id'].isin(active_users)]
```

```
# Use parent_asin as item ID everywhere
merge_df['item_id'] = merge_df['parent_asin']    # single canonical item column
```

Expected output after filter: ~180-200K rows, ~15-25K unique users with ≥ 5 interactions, ~38-47K unique products (parent_asin).

1.5 New metadata fields to use (previously unused)

Field	Where to use	Why it improves the project
features (list)	File 07 — product_text	Bullet-point product attributes. Join with space and add to product_text before encoding. Makes semantic search queries like 'acoustic guitar spruce top' return correct results.
details (dict)	File 07 — product_text	Brand, material, size, connectivity. str(details) added to product_text. Greatly improves e5-base-v2 embedding distinctiveness between similar products.
bought_together (list)	File 04 — Apriori validation	Amazon's own cross-sell bundles. Use to validate your Apriori rules: if your rule (A->B) also appears in bought_together, it is ground-truth confirmed. Show this in A/B notebook.
price (float)	File 05 — ProductRecommender	Add price as a scoring dimension in satisfaction_score. Penalise outlier prices. Previously dropped — was a missed signal.
helpful_vote (int)	File 05 — ProductRecommender	Already used but now weight it higher — helpful votes in Software and Musical Instruments are a strong quality signal because buyers are technical and vote carefully.

1.6 Environment — where each task runs

Task	Anaconda local (6GB VRAM)	Colab (one-time only)	Where
All data loading, preprocessing, CF models, Apriori, MLflow, FastAPI, Docker, DVC	Everything runs here. Full RAM, no session timeout, Git + DVC + Docker work.	—	Local
e5-base-v2 encoding (700K rows → review_embeds.npy, meta_embeds.npy)	Possible with batch_size=8 + model.half() (fp16). Takes 3-4 hrs.	Recommended. Free A100/T4. ~20 min. Save .npy to Drive, download to local embeddings/.	Colab once
FastAPI + Docker + DVC + MLflow UI	Must run locally. Cannot expose ports or run Docker in Colab.	Not possible.	Local only

2. System Overview — All Requirements Mapped

Every DS11 requirement maps to a specific file and dataset field. This table is the ground truth for what satisfies the assignment.

DS11 Requirement	How satisfied
TF-IDF content features	TfidfVectorizer(15K features, ngram (1,2)) on text field — feeds LinearSVC sentiment classifier
Sentence embeddings	e5-base-v2 encodes product_text = title+description+features+details+categories per parent_asin
Matrix Factorization — ALS	implicit.als.AlternatingLeastSquares(factors=64, iterations=20) on parent_asin user-item matrix
SVD++	Surprise SVDpp(n_factors=50, n_epochs=20) on same user-parent_asin-rating triplets
Hybrid fusion — weighted	HybridRecommender unions top-20 from Apriori + CF + ProductRecommender with configurable weights
Cold-start handling	New user: content-only path (ProductRecommender + semantic search). New item: embedding similarity only.
MLflow experiment tracking	7 runs logged: SVM, content_only, MF, NCF, ALS, SVDpp, Hybrid. Same experiment 'DS11'.
DVC data pipeline	3 stages: ingest → preprocess → embed. dvc repro reproducible end-to-end.
FastAPI service	POST /recommend, GET /similar/{asin}, GET /search, GET /health
Docker container	Dockerfile + docker-compose.yml. docker-compose up starts service on :8000.
A/B comparison notebook	Content-only vs ALS-only vs MF/NCF vs Hybrid on same test split. Cold-start section. Summary table.
MLflow experiment report	mlflow.search_runs() → param table → metric leaderboard → artifact links → exported HTML.
BONUS: Apriori market-basket	mlxtend Apriori on user-parent_asin baskets. Validated against bought_together ground truth.
BONUS: NCF (Neural CF)	GMF + MLP [64,32] PyTorch model alongside MF.

BONUS: Semantic search + reranking	e5+BM25 hybrid, 4-band clustering, quality/trending/personalized/relevance rerankers.
BONUS: T5 summarization	T5-small abstractive summaries for top-500 products. outputs/summary.csv.
BONUS: User profile embeddings	Top-50 user vectors for personalized reranking in semantic search.

3. Complete Step-by-Step Workflow

Run phases 0 → 12 in order. Tag legend below. Orange = new addition or fix required.

Core requirement

MLOps

API / Deploy

Extra / Bonus

Data pipeline

NEW / Fix

PHASE 0

Environment Setup

#	Action	Detail / Output	Tag
0.1	Create project folder	amazon_project/ with src/, data/, embeddings/, models/, api/, mlflow/, outputs/	Data
0.2	Install core packages	pip install pandas numpy scikit-learn torch transformers sentence-transformers mlxtend rank_bm25 tqdm dill wordcloud	Core
0.3	Install ALS + SVD++ packages	pip install implicit scikit-surprise — required for file 09	NEW
0.4	Install MLOps + API packages	pip install mlflow dvc fastapi uvicorn python-multipart	NEW
0.5	Create config.py	CATEGORIES=["Video_Games","Musical_Instruments","Software"], HF_BASE_URL, CHUNK_SIZE=100000, all dir paths, DEVICE, MLFLOW_URI	Core
0.6	Initialise DVC repo	dvc init; git add .dvc .dvcignore	NEW
0.7	Set MLflow tracking URI	mlflow.set_tracking_uri('mlflow/') in config; mlflow.set_experiment('DS11')	NEW

PHASE 1

Data Ingestion & Merging

#	Action	Detail / Output	Tag
1.1	Load 3 categories	Video_Games, Musical_Instruments, Software — CHUNK_SIZE=100000 each from HuggingFace	Core
1.2	Add main_category column	Tag each chunk before concat	Core
1.3	Concat review_df and meta_df	pd.concat both lists; delete per-category lists immediately to free RAM	Core
1.4	Merge on parent_asin = parent_asin	review_df.merge(meta_df, left_on='parent_asin', right_on='parent_asin', suffixes=('_rev','_meta'))	Fix
1.5	Add item_id column	merge_df['item_id'] = merge_df['parent_asin'] — single canonical item ID for all CF work	Fix
1.6	Drop unmatched rows	dropna(subset=['parent_asin_meta']); log rows removed and %	Core
1.7	Apply user-activity filter	Keep only users with >= 5 interactions: user_counts[user_counts>=5].index	Fix
1.8	Log filter stats	print rows/users/items before and after filter; verify avg interactions per user > 5	Core
1.9	Save to CSV + parquet	merge_df.to_csv('data/merge_df.csv'); merge_df.to_parquet('data/clean_merge_df.parquet')	Data

1.1 0	Register DVC ingest stage	dvc run -n ingest -d src/01_data_ingestion.py -o data/merge_df.csv	NEW
----------	---------------------------	--	-----

PHASE 2 Preprocessing & Feature Engineering

#	Action	Detail / Output	Tag
2.1	Null audit	Plot % nulls per column; record original_shape and na_before	Core
2.2	Drop high-null columns	Drop cols where null% > 40%	Core
2.3	Impute remaining nulls	rating_number -> column mean; store -> 'Unknown'; price -> median	Core
2.4	Cast types	main_category -> Categorical; timestamp -> datetime; item_id stays string	Core
2.5	Time-of-day binning	pd.cut on timestamp.dt.hour -> night/morning/midday (ordered=False)	Core
2.6	Date features	year, month, day, dayofweek, is_weekend from timestamp	Core
2.7	Clean review text	Lowercase, strip URLs, keep [a-z\s] -> text_clean	Core
2.8	Build product_text	title_meta + '' + description + '' + ''.join(features) + '' + str(details) + '' + str(categories) per item_id — includes new fields	Fix
2.9	Build full_review_text	title_rev + '' + text per row for review-level embedding	Core
2.1 0	Print preprocessing report	Shape/nulls before+after; columns added/removed	Core
2.11	Register DVC preprocess stage	dvc run -n preprocess -d src/02_preprocessing.py -d data/merge_df.csv -o data/clean_merge_df.parquet	NEW

PHASE 3 Sentiment Analysis & NLP

#	Action	Detail / Output	Tag
3.1	Create sentiment labels	rating<=2 -> negative; 3 -> neutral; >=4 -> positive	Core
3.2	TF-IDF vectorize	TfidfVectorizer(max_features=15000, ngram_range=(1,2)) on text_clean	Core
3.3	Train LinearSVC	80/20 split; print classification_report + confusion_matrix	Core
3.4	Log to MLflow	mlflow.start_run(run_name='SVM'): log max_features, ngram_range, accuracy, f1_macro	NEW
3.5	Persist SVM	pickle.dump(vectorizer + svm_model) to models/	Core
3.6	T5 summarization (BONUS)	top-500 products by review count; group first 50 reviews; T5-small pipeline max_length=120	Extra
3.7	Export summary_df	outputs/final_top500_product_summary.csv with asin, summary, avg_sentiment, n_reviews	Extra
3.8	Visualize	Sentiment bar, rating vs sentiment scatter, word cloud — Software has richest vocabulary	Core

PHASE 4

Apriori Market-Basket (BONUS)

#	Action	Detail / Output	Tag
4.1	Build baskets on item_id	groupby user_id -> list of item_id (parent_asin); keep basket size 3-12	Extra
4.2	One-hot encode	mlxtend TransactionEncoder -> boolean DataFrame	Extra
4.3	Run Apriori	min_support=0.0005; association_rules(metric='lift', min_threshold=1.0)	Extra
4.4	Keep 1->1 rules only	Filter antecedent size=1 AND consequent size=1	Extra
4.5	Validate against bought_together	For each rule A->B, check if B appears in meta bought_together for A. Log % confirmed by ground truth.	Extra
4.6	Build AprioriRecommender class	recommend_apriori(item_id, top_k) checks both directions; verify_recommendations() adds title_meta	Extra

PHASE 5

Content-Based + Item-Item CF

#	Action	Detail / Output	Tag
5.1	Aggregate product features	Per item_id: rating_mean, review_count, helpful_votes, verified_count, last_review_date, price_median	Core
5.2	Compute composite scores	satisfaction = 0.6*rating + 0.2*verified_rate + 0.2*log(helpful); recency; popularity; hotness	Core
5.3	Add price to scoring	price_score = 1 / (1 + abs(price - category_median_price) / category_median_price)	Fix
5.4	ProductRecommender class	get_recommendations(item_id, top_n, weights) with 5 scoring components	Core
5.5	Build sparse rating matrix	csr_matrix (user_idx, item_idx) using item_id — NOT asin — as item index	Fix
5.6	Mean-center by user	Subtract per-user mean rating to remove rating-scale bias	Core
5.7	Compute top-50 item neighbors	Cosine sim via sparse multiply; argpartition for speed	Core
5.8	CollaborativeFilteringRecommender	Encapsulates prep + neighbor compute + recommend_products_cf(item_id, top_k)	Core
5.9	Log content-only run to MLflow	mlflow.start_run(run_name='content_only'): log weights dict, Recall@10, NDCG@10 on test users	NEW

PHASE 6

MF + NCF (PyTorch)

#	Action	Detail / Output	Tag
6.1	Load CF data	clean_merge_df.parquet; select user_id, item_id, rating, timestamp	Core
6.2	Encode IDs to integers	user_map + item_map on item_id (parent_asin); save as JSON	Core

6.3	Temporal train/test split	Sort by timestamp; last interaction per user -> test	Core
6.4	Build RatingDataset + DataLoader	batch=4096, shuffle=True, pin_memory=True	Core
6.5	Define MF model	user_emb + item_emb (emb_dim=64) + biases; dot product — fits in ~1GB VRAM	Core
6.6	Define NCF model (BONUS)	GMF path + MLP [64,32] concatenated; ~1.5GB VRAM	Extra
6.7	Train both models	Adam + MSELoss; print RMSE per epoch; 8 epochs MF, 10 epochs NCF	Core
6.8	Evaluate RMSE + Recall@10 + NDCG@10	500 sampled negatives per user on same test split	Core
6.9	Log MF run to MLflow	mlflow.start_run(run_name='MF'): emb_dim, epochs, lr, RMSE, Recall@10, NDCG@10	NEW
6.10	Log NCF run to MLflow	mlflow.start_run(run_name='NCF'): same metric set	NEW
6.11	Save weights + metrics	models/mf_model.pth, ncf_model.pth, metrics.json	Core
6.12	UMAP visualization (BONUS)	UMAP of item embeddings; MF vs NCF metric bar chart	Extra

PHASE 7

Semantic Search + Reranking (BONUS)

#	Action	Detail / Output	Tag
7.1	Build product_text	title_meta + description + ' '.join(features) + str(details) + str(categories) — use new fields	Fix
7.2	Encode reviews (Colab / local fp16)	e5-base-v2 batch=8 (local) or batch=64 (Colab); normalize=True; save review_embeddings.npy	Core
7.3	Encode meta per item_id	Group by item_id (parent_asin); first product_text; save meta_embeddings.npy	Core
7.4	Build product_vecs dict	>=3 reviews: 0.7*rev_mean + 0.3*meta; else swap; normalize; key = item_id	Core
7.5	Build BM25 index	Tokenize bm25_text per item_id; BM25Okapi fit	Extra
7.6	Hybrid retrieval	0.55*emb + 0.45*bm25; threshold=0.20; min 20 results	Extra
7.7	Score-band clustering	4 bands by similarity; centroid cosine ordering	Extra
7.8	Reranking modes	quality / trending / personalized / relevance	Extra
7.9	User profile embeddings	Top-50 users; rating-weighted review text; encode; save top_user_embeddings.npy	Extra
7.10	Register DVC embed stage	dvc run -n embed -d src/07_semantic_search.py -o embeddings/	NEW

PHASE 8

Hybrid Engine + Cold-Start

#	Action	Detail / Output	Tag
8.1	HybridRecommender class	Wraps Apriori + CF + ProductRecommender; union of top-20 from each engine	Core

8.2	Cold-start: new user	If user_id not in CF history -> route to ProductRecommender + semantic search only	NEW
8.3	Cold-start: new item	If item has < 3 reviews -> product_text embedding similarity only	NEW
8.4	final_recommendation(item_id, user_id)	Detects cold-start automatically; returns metadata with source tag per result	Core
8.5	Log Hybrid run to MLflow	mlflow.start_run(run_name='Hybrid'): cf_weight, content_weight, Recall@10, NDCG@10, Precision@10	NEW
8.6	Serialize with dill	dill.dump(hybrid_recommender, 'models/hybrid_recommender.pkl')	Core

PHASE 9

ALS + SVD++ Models — NEW FILE

#	Action	Detail / Output	Tag
9.1	Build item-user matrix	scipy csr_matrix in item-user orientation (Implicit library expects transposed). Use item_id not asin.	NEW
9.2	Train ALS	AlternatingLeastSquares(factors=64, iterations=20, regularization=0.1, use_gpu=False)	NEW
9.3	Train SVD++	Surprise SVDpp(n_factors=50, n_epochs=20) on same triplets	NEW
9.4	Evaluate both	RMSE, Recall@10, NDCG@10 on same test split as file 06 — required for fair A/B comparison	NEW
9.5	Log ALS to MLflow	mlflow.start_run(run_name='ALS'): factors, iterations, regularization, RMSE, Recall@10, NDCG@10	NEW
9.6	Log SVDpp to MLflow	mlflow.start_run(run_name='SVDpp'): n_factors, n_epochs, RMSE	NEW
9.7	Save models	models/als_model.npz (implicit save); models/svdpp_model.pkl (pickle)	NEW

PHASE 10

A/B Comparison Notebook — NEW FILE

#	Action	Detail / Output	Tag
10.1	Load all models + same test split	Load test_df saved from file 06 — same split for all systems	NEW
10.2	Section 2: Content-only baseline	ProductRecommender.get_recommendations() on test users; Recall@10, NDCG@10	NEW
10.3	Section 3: ALS-only baseline	ALS model.recommend() on test users; same metrics	NEW
10.4	Section 4: MF + NCF	PyTorch model predictions; same metrics	NEW
10.5	Section 5: Hybrid	hybrid_recommender.final_recommendation(); same metrics	NEW
10.6	Section 6: Cold-start simulation	Zero out history for 100 test users; re-run all systems; show metric delta	NEW

10.7	Section 7: Apriori validation	Show % of Apriori rules confirmed by bought_together ground truth	Extra
10.8	Section 8: Summary table + charts	DataFrame [system x metric]; grouped bar charts; annotate winner per metric	NEW
10.9	Export	outputs/ab_comparison_results.csv	NEW

PHASE 11

MLflow Experiment Report — NEW FILE

#	Action	Detail / Output	Tag
11.1	Load all 7 runs	mlflow.search_runs(experiment_names=['DS11']) -> DataFrame	NEW
11.2	Param comparison table	All runs x all params side by side	NEW
11.3	Metric leaderboard	Sort by NDCG@10 desc; bold winner per column	NEW
11.4	Artifact links	confusion_matrix.png, classification_report.txt, UMAP plot	NEW
11.5	Export HTML	jupyter nbconvert --to html -> outputs/mlflow_report.html	NEW

PHASE 12

FastAPI Service — NEW FILE

#	Action	Detail / Output	Tag
12.1	api/main.py — startup	Load hybrid_recommender.pkl + product_vecs + als_model on lifespan event	NEW
12.2	POST /recommend	Body: {item_id, user_id, top_k=10} -> [{item_id, title, score, source}, ...]	NEW
12.3	GET /similar/{item_id}	Top-K similar items by embedding similarity	NEW
12.4	GET /search?q=	Calls semantic search; returns ranked results with scores	NEW
12.5	GET /health	Returns {status: ok, model_loaded: true}	NEW
12.6	api/schemas.py	Pydantic request/response models; 422 on bad inputs	NEW
12.7	api/Dockerfile	FROM python:3.10-slim; COPY; pip install; EXPOSE 8000; CMD uvicorn	NEW
12.8	api/docker-compose.yml	Service definition; volume mount models/ and embeddings/; port 8000:8000	NEW
12.9	api/test_api.py	3+ end-to-end tests using httpx: /health, /recommend, /search	NEW

4. Complete Project File Structure

```
amazon_project/
├── config.py      # CATEGORIES, paths, MLflow URI, DEVICE
├── dvc.yaml       # NEW — 3 stages: ingest, preprocess, embed
├── README.md      # Setup, docker-compose up, sample curl calls
├── data/
│   ├── merge_df.csv      # 300K raw rows (before user filter)
│   ├── clean_merge_df.parquet  # ~180-200K after filter (DVC output)
│   ├── cleaned_cf_dataset.parquet  # user_id, item_id, rating, timestamp only
│   ├── user_map.json      # user_id -> int
│   └── item_map.json      # item_id (parent_asin) -> int
├── embeddings/
│   ├── review_embeds.npy  # Per-row review vectors (768d) — Colab generated
│   ├── meta_embeds.npy    # Per-item_id meta vectors (768d)
│   └── top_user_embeddings.npy  # Top-50 user profile vectors
├── models/
│   ├── mf_model.pth        # Matrix Factorization weights
│   ├── ncf_model.pth       # Neural CF weights (BONUS)
│   ├── als_model.npz       # NEW — ALS model (Implicit)
│   ├── svdpp_model.pkl     # NEW — SVD++ model (Surprise)
│   ├── hybrid_recommender.pkl  # Full hybrid engine (dill serialized)
│   ├── svm_vectorizer.pkl  # TF-IDF vectorizer
│   ├── svm_model.pkl       # LinearSVC sentiment classifier
│   └── metrics.json        # All model RMSE / Recall@10 / NDCG@10
├── mlflow/         # NEW — MLflow tracking store
│   └── mlruns/      # Auto-created; 7 runs inside
├── outputs/
│   ├── final_top500_product_summary.csv
│   ├── ab_comparison_results.csv  # NEW — A/B deliverable output
│   └── mlflow_report.html         # NEW — MLflow report deliverable
├── src/
│   ├── 01_data_ingestion.py      # Phase 1      (parent_asin fix)
│   ├── 02_preprocessing.py       # Phase 2      (new fields: features, details)
│   ├── 03_sentiment_nlp.py       # Phase 3      (+ MLflow SVM run)
│   ├── 04_apriori_recommender.py  # Phase 4      (BONUS + bought_together validation)
│   ├── 05_content_cf_recommender.py  # Phase 5      (+ price score + MLflow run)
│   ├── 06_mf_ncf_pytorch.py      # Phase 6      (+ MLflow MF+NCF runs)
│   ├── 07_semantic_search.py     # Phase 7      (BONUS + richer product_text)
│   ├── 08_hybrid_engine.py       # Phase 8      (+ cold-start + MLflow run)
│   ├── 09_als_svdpp.py           # Phase 9 NEW
│   ├── 10_ab_comparison.ipynb    # Phase 10 NEW (graded deliverable)
│   └── 11_mlflow_report.ipynb    # Phase 11 NEW (graded deliverable)
├── api/             # NEW — FastAPI service (graded deliverable)
│   ├── main.py       # App + all endpoints
│   ├── schemas.py    # Pydantic models
│   ├── test_api.py   # End-to-end tests
│   ├── Dockerfile    # python:3.10-slim, EXPOSE 8000
│   └── docker-compose.yml  # Service + volume mounts
```

5. File-by-File Breakdown — Changes & New Files

Files 01–08 need small targeted changes (shown below). Files 09–11 and api/ are fully new.

Changes to existing files 01–08

File	Change required	Specific line / location
01_data_ingestion.py	3 changes: (1) CATEGORIES to 3 only, (2) merge on parent_asin=parent_asin, (3) user-activity filter replaces random sample	merge line + filter line after dropna + add DVC stage call
02_preprocessing.py	Add features+details+price to product_text builder. Add price imputation. Register DVC preprocess stage.	product_text concat block; imputation block; bottom of file
03_sentiment_nlp.py	Add 5 MLflow lines: set_tracking_uri, set_experiment, start_run, log_param x2, log_metric x2	After model.fit(); before pickle.dump()
04_apriori_recommender.py	Use item_id instead of asin for baskets. Add bought_together validation step.	groupby line + new validate_against_bought_together() function
05_content_cf_recommender.py	Add price_score component to ProductRecommender. Use item_id not asin for CF matrix. Add MLflow content_only run.	_aggregate_product_features(); csr_matrix build; end of file
06_mf_ncf_pytorch.py	Use item_id (parent_asin) for item_map — not asin. Add MLflow MF and NCF runs.	item_map line; after evaluate_rmse() for both models
07_semantic_search.py	Update product_text to include features, details fields. Use item_id as dict key throughout. Add DVC embed stage.	product_text build block; all product_vecs[asin] -> product_vecs[item_id]
08_hybrid_engine.py	Add explicit cold-start routing. Add MLflow Hybrid run. Accept user_id in final_recommendation().	final_recommendation() signature + cold-start if-block + MLflow block

File 9 — 09_als_svdpp.py (NEW)

09_als_svdpp.py — <i>ALS via Implicit + SVD++ via Surprise — closes the required MF algorithm gap</i>		NE W
Owner	Utkrishta	
Depends On	data/cleaned_cf_dataset.parquet, data/user_map.json, data/item_map.json	
Outputs	models/als_model.npz, models/svdpp_model.pkl — MLflow runs: ALS, SVDpp	

Key implementation notes:

- Implicit expects item-user matrix (transposed from user-item). If recs are garbage, check this first.
- ALS with factors=64 on ~180K interactions runs in ~5 minutes on CPU — no GPU needed.
- SVD++ n_epochs=20 takes ~15-20 min on CPU. Reduce to 10 if slow.
- Both must use item_id (parent_asin) as item index — not asin. Same map as file 06.
- Evaluate on the exact same test_df saved from file 06. Different test splits make A/B comparison meaningless.

File 10 — 10_ab_comparison.ipynb (NEW — Graded Deliverable)

10_ab_comparison.ipynb — <i>A/B comparison: content-only vs ALS-only vs MF/NCF vs Hybrid on same metrics</i>		NE W
Owner	Everyone	
Depends On	All saved models; test_df from file 06; bought_together from meta for Apriori validation	
Outputs	outputs/ab_comparison_results.csv	

Notebook structure — 9 sections:

1. Setup — load all models, define shared evaluate(system, test_df) function, define metrics: RMSE, Recall@10, NDCG@10, Precision@10
2. Content-only — ProductRecommender on test users
3. ALS-only — ALS model.recommend() on test users
4. MF + NCF — PyTorch predictions on test users
5. Hybrid — HybridRecommender.final_recommendation() on test users
6. Cold-start simulation — zero out history for 100 users; re-run all systems; delta table shows which handles cold-start best
7. Apriori validation — % of rules confirmed by bought_together ground truth (bonus insight)
8. Summary table + grouped bar charts — one DataFrame, all systems x all metrics, annotate winner
9. Export — results.to_csv('outputs/ab_comparison_results.csv')

File 11 — 11_mlflow_report.ipynb (NEW — Graded Deliverable)

11_mlflow_report.ipynb — <i>Pull all 7 MLflow runs; generate the experiment report HTML</i>		NE W
Owner	Everyone	
Depends On	mlflow/mlruns/ (populated by files 03, 05, 06, 08, 09)	

Outputs	outputs/mlflow_report.html	
---------	----------------------------	--

Run order matters — all 7 runs must be logged before running this file:

- Run file 03 first (SVM run), then 05 (content_only), then 06 (MF + NCF), then 09 (ALS + SVDpp), then 08 (Hybrid)
- All must use: `mlflow.set_tracking_uri('mlflow/')` and `mlflow.set_experiment('DS11')`
- Final cell: `jupyter nbconvert --to html src/11_mlflow_report.ipynb --output ../outputs/mlflow_report.html`

api/main.py + Dockerfile (NEW — Graded Deliverable)

api/ — FastAPI service — POST /recommend, GET /similar, GET /search, GET /health		NEW
Owner	Everyone	
Depends On	models/hybrid_recommender.pkl; embeddings/; models/als_model.npz	
Outputs	Running service on :8000 — tested with api/test_api.py	

Four endpoint signatures:

```
POST /recommend      {item_id: str, user_id: str, top_k: int=10}
GET  /similar/{item_id}?top_k=10
GET  /search?q=wireless+headphones&top_k=10
GET  /health         -> {status: ok, model_loaded: bool}
```

Cold-start detection in /recommend:

```
if user_id not in cf_recommender.asin_to_item_idx:
    return content_only_path(item_id, top_k)  # new user
if item_reviews[item_id] < 3:
    return embedding_similarity_path(item_id, top_k)  # new item
```

6. MLflow Integration — Exact Pattern

Add these lines to each training file. All use the same experiment name so file 11 can pull all runs together.

```
import mlflow
mlflow.set_tracking_uri('mlflow/') # relative path — works from any src/ file
mlflow.set_experiment('DS11') # same name in every file

with mlflow.start_run(run_name='MF'):
    mlflow.log_param('emb_dim', 64)
    mlflow.log_param('epochs', 8)
    mlflow.log_param('lr', 1e-3)
    # ... train ...
    mlflow.log_metric('rmse', mf_rmse)
    mlflow.log_metric('recall_at_10', mf_recall)
    mlflow.log_metric('ndcg_at_10', mf_ndcg)
    mlflow.log_artifact('models/mf_model.pth')
```

File	Run name	Params to log	Metrics to log
03	SVM	max_features, ngram_range	accuracy, f1_macro, f1_weighted
05	content_only	weights dict, top_n	recall_at_10, ndcg_at_10, precision_at_10
06	MF	emb_dim=64, epochs=8, lr=1e-3, batch=4096	rmse, recall_at_10, ndcg_at_10
06	NCF	emb_dim=32, mlp=[64,32], epochs=10, lr=1e-3	rmse, recall_at_10, ndcg_at_10
09	ALS	factors=64, iterations=20, regularization=0.1	rmse, recall_at_10, ndcg_at_10
09	SVDpp	n_factors=50, n_epochs=20	rmse, recall_at_10, ndcg_at_10
08	Hybrid	cf_weight, content_weight, apriori_weight	recall_at_10, ndcg_at_10, precision_at_10

7. Debugging Quick-Reference

Symptom	Likely cause	Fix
ALS recs all from one category	Matrix is user-item; Implicit needs item-user	Transpose: user_item_matrix.T before passing to ALS.fit()
CF item neighbors all within same product	Still using asin (variant) not parent_asin	Check item_id column exists and is parent_asin. Rerun file 01 with the fix.
Apriori returns 0 rules	Baskets still have single items (random sample remnant)	Verify user-activity filter ran. print(transactions.apply(len).describe()) — mean should be >3

MLflow runs not in file 11	Different tracking URI or experiment name	Every file must call <code>set_tracking_uri('mlflow/') AND set_experiment('DS11')</code> before <code>start_run</code>
e5-base-v2 CUDA OOM locally	Batch too large for 6GB VRAM	<code>model.half() + batch_size=8</code> . Or use Colab for this step only and download .npz files.
FastAPI 500 on startup	Pickle path wrong inside container	<code>docker-compose</code> volume mount must match path in <code>main.py</code> . Use absolute <code>/app/models/</code> paths inside container.
A/B metrics inconsistent across systems	Different test splits used	All systems must use same <code>test_df</code> . Save it as <code>data/test_df.parquet</code> in file 06 and load it in file 10.
<code>product_vecs</code> key not found	Mixed <code>item_id</code> / <code>asin</code> / <code>parent_asin</code> keys	Standardise: everywhere use <code>merge_df['item_id']</code> which equals <code>parent_asin</code> . Check file 07 uses <code>item_id</code> as dict key.
SVD++ very slow	Running on full 200K rows	Surprise is CPU-only and slow. Reduce <code>n_epochs=10</code> . Or sample 50K rows for SVD++ only — note it in MLflow params.
dvc repro fails	Stage output already exists but hash changed	<code>dvc repro --force</code> to ignore cache. Run <code>dvc status</code> first to see what changed.
Cold-start returns nothing	<code>user_id</code> type mismatch (str vs int)	<code>print(type(user_id))</code> vs type of keys in <code>cf_recommender</code> dict. Ensure both are str throughout.

8. Final Delivery Checklist

Graded deliverable	What to submit / verify	File
Hybrid recommendation API	<code>docker-compose</code> up starts service; <code>curl POST /recommend</code> returns product list; <code>GET /health</code> returns ok	<code>api/</code>
A/B comparison notebook	<code>10_ab_comparison.ipynb</code> fully run with all outputs; <code>outputs/ab_comparison_results.csv</code> exists	<code>src/10</code>
MLflow experiment report	<code>outputs/mlflow_report.html</code> exists; all 7 runs visible in mlflow ui (<code>mlflow ui --backend-store-uri mlflow/</code>)	<code>src/11</code>
ALS model (spec requires)	<code>models/als_model.npz</code> exists; MLflow ALS run logged with factors, iterations, RMSE, Recall@10	<code>src/09</code>
SVD++ model (spec requires)	<code>models/svdpp_model.pkl</code> exists; MLflow SVDpp run logged	<code>src/09</code>
Cold-start handling (spec requires)	Documented and tested in file 10 Section 6. Both new-user and new-item paths in file 08.	<code>src/08+10</code>
DVC data pipeline (spec requires)	<code>dvc repro</code> runs clean end-to-end from raw download to embeddings	<code>dvc.yaml</code>

parent_asin fix applied everywhere	grep -r 'left_on="asin"' src/ returns nothing. All CF uses item_id column.	src/01,05,06,07,08,09
User-activity filter (not random sample)	grep -r 'sample(' src/ returns nothing. Filter code uses user_counts[user_counts>=5].	src/01
BONUS: Semantic search in API	GET /search?q=... returns relevant results from 3-category domain	api/ + src/07
BONUS: T5 + Apriori + NCF + user embeddings	outputs/summary.csv exists; Apriori rules logged with bought_together validation %	src/03,04,06,07

Dataset: Amazon Reviews 2023 — Video_Games + Musical_Instruments + Software only. Merge on parent_asin. User-activity filter (>=5 interactions). No MovieLens. No random sampling.